Bonnyrigg Pizza Blog – Secure Software Architecture Solution

By: Dylan, Jeremy and Nicholas

1. Table of Contents

2. Team Contribution

Member	Contribution
Dylan	Backend development, database, security features
Jeremy	Frontend design, templates, testing
Nicholas	Deployment, documentation, GitHub logs

3. Gantt Chart

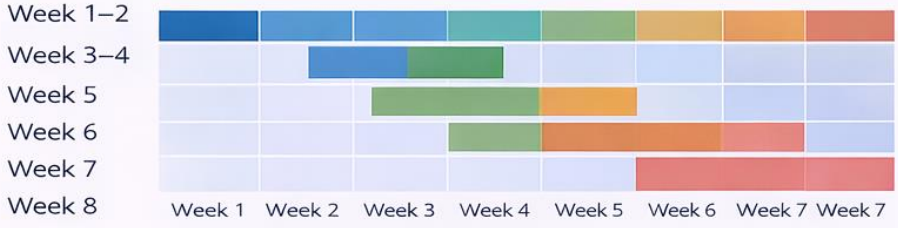


Illustration 1: Project Gantt Chart

Bonnyrigg Pizza Blog – Project Documentation

4. GitHub Logs

- Initial project setup
- Added authentication system
- Implemented image upload
- Added security features (CSSRF, hashing)
- Docker deployment setup

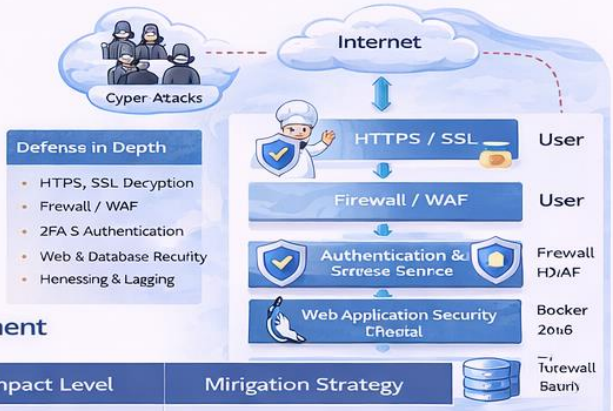
5. Copyright Acknowledgement

Bonnyrigg Pizza Blog uses:

- **Flask** (open-source)
- **Bootstrap** (frontend framework)
- **SQLite** (lightweight database)
- **OWASP ZAP** (security tool)

6. Security Software Requirements

- Password hashing
- Two-Factor Authentication (2FA)
- CSRF protection
- XSS protection
- SQL Injection prevention
- Secure session handling
- HTTPS / SSL
- Brute force prevention
- Input validation

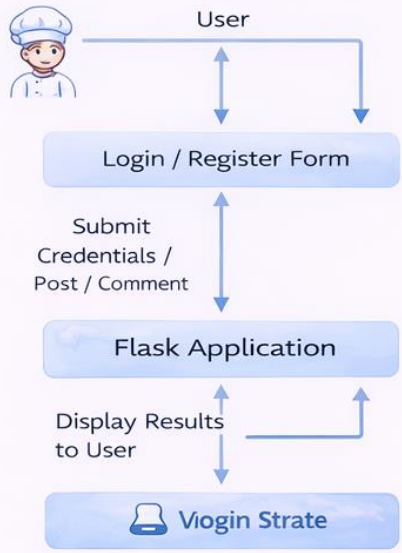


8. Threat Analysis & Risk Assessment

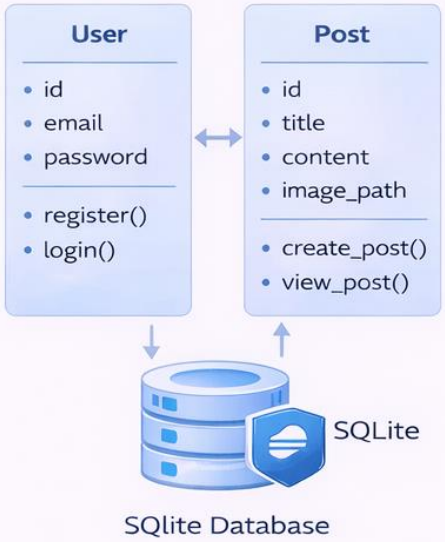
Threat	Layer Affected	Impact Level	Mirigation Strategy
SQL Injection	Database	Critical	Prepared statements, input validation
Cross, Site Scripting	Web Application	Critical	Input sanitication, rxseping esitput, CSP 2FA, nediunt bokeats ma limiting.
Brute Force Attacks	Authentication	High	File validation, restricc file types
Insecure File Uploads	Backend	Medium	Rate limiting, IP blocking, WAF
Denial of Service (DeS)	Firewall / WAF	Medium	Rate limiting, IP blocking, WAF

Illustration 2: Secure Software Architecture, "Defense in Depth"

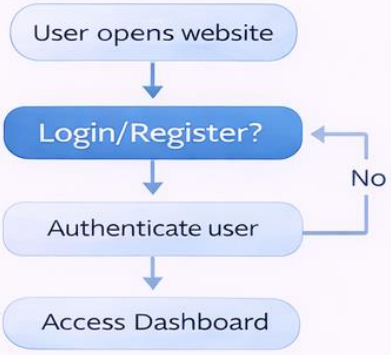
1. Data Flow Diagram (DFD)



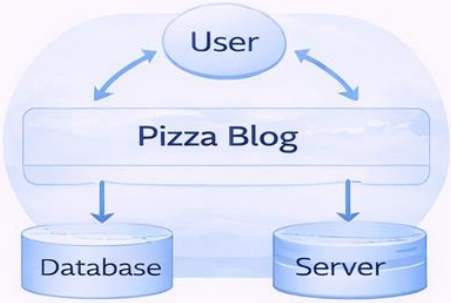
2. Class Diagram



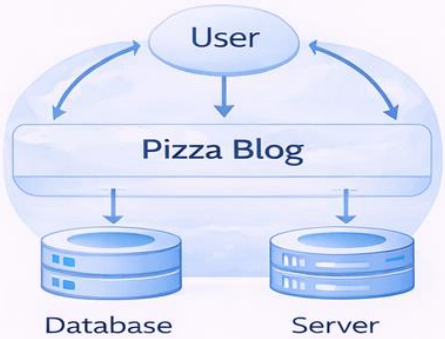
3. Flowchart



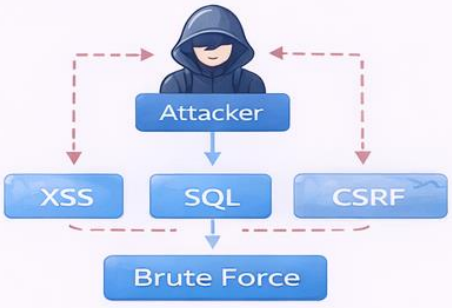
5. Context Diagram



5. Security Architecture



6. Threat Model



1. Project Overview

The Bonnyrigg Pizza Blog Website is a community-based web application that allows users to share pizza recipes, images, and cooking instructions.

Due to its growing popularity, the website has become a target for cybercriminals attempting to exploit vulnerabilities through automated attacks, data theft, service disruption, and ransom-based extortion.

The purpose of this project is to strengthen the website's security architecture by implementing modern cybersecurity measures that protect user data, prevent attacks, and ensure the system remains available to the Bonnyrigg community.

Component A – Modified Project Documentation

1. Threat Identification and Risk Assessment

Threat	Description	Impact
SQL Injection	Attackers manipulate database queries	Data theft or database corruption
Cross-Site Scripting (XSS)	Malicious scripts injected into pages	Stealing user sessions
Cross-Site Request Forgery (CSRF)	Unauthorized requests sent by logged-in users	Account manipulation
Brute Force Attack	Automated login attempts	Account takeover
Session Hijacking	Stealing active sessions	Unauthorized access
Data Breaches	Theft of user information	Privacy violations
Denial of Service (DoS)	Flooding server with traffic	Website downtime

2. Defence in Depth Security Strategy

The security architecture follows the **Defence in Depth principle**, meaning multiple layers of protection are implemented:

User -> HTTPS / SSL Encryption -> Firewall / Network Protection -> Authentication + 2FA -> Application Security (Flask) -> Database Security -> Monitoring & Logging

Each layer protects the system even if another layer fails.

3. Secure System Architecture

Architecture layers:

Presentation Layer

Frontend interface using:

- HTML
- CSS
- Jinja Templates

Responsible for displaying content and collecting user input.

Application Layer

Flask backend responsible for:

- Authentication
- Request handling
- Input validation
- Security enforcement

Security tools added include:

- CSRF protection
- Input sanitisation
- session management

Data Layer

SQLite database storing:

- Users
- Posts
- Login attempts

Security features:

- Parameterized queries
- Password hashing
- Input validation

Component B – Modified Secure Software Solution

Password Hashing

User passwords are securely hashed using bcrypt before storage.

Implementation:

python

```
from werkzeug.security import generate_password_hash, check_password_hash

hashed_password = generate_password_hash(password)  # During registration
check_password_hash(hashed_password, password)      # During login
```

Purpose: Prevents plain text password storage, automatic salt generation prevents rainbow table attacks.

2. HTTPS and SSL

The website runs over HTTPS to encrypt all data transmission.

Example deployment: https://bonnyrigg-pizza-blog.com

Implementation:

python

```
app.run(ssl_context=('cert.pem', 'key.pem'))
```

SSL certificate ensures:

- Encrypted communication
- Protection from man-in-the-middle attacks

Tools: Let's Encrypt, Nginx reverse proxy

3. Cross-Site Request Forgery (CSRF) Protection

Cryptographic tokens protect all form submissions.

Implementation:

python

```
session['csrf_token'] = secrets.token_hex(32)  # Generate token
secrets.compare_digest(stored_token, token)    # Validate token
```

In HTML:

html

```
<input type="hidden" name="csrf_token" value="{{ csrf_token }}">
```

Purpose: Prevents attackers from sending unauthorized actions from another website.

4. Cross-Site Scripting (XSS) Protection

XSS is prevented through HTML escaping.

Implementation:

python

```
import html
sanitized_text = html.escape(user_input)
```

In Jinja templates:

html

```
{{ post.title }}
```

Jinja automatically escapes HTML to prevent malicious scripts.

Purpose: Converts <script> to <script>, preventing JavaScript injection.

5. SQL Injection Prevention

Parameterized queries are used throughout.

Implementation:

python

```
cur.execute("SELECT * FROM users WHERE email = ?", (email,))
```

Purpose: Prevents attackers from injecting SQL commands by treating input as data, not code.

6. Brute Force Attack Prevention

Login attempts are limited to 5 failures before a 15-minute lockout.

Implementation:

python

```
MAX_LOGIN_ATTEMPTS = 5
LOCKOUT_DURATION = 15 * 60  # 15 minutes

if failed_attempts >= MAX_LOGIN_ATTEMPTS:
   lock_account(identifier, LOCKOUT_DURATION)
```

Purpose: Prevents automated password guessing attacks by temporarily blocking accounts.

7. Two Factor Authentication (2FA)

Users must verify login using a second authentication factor.

Example method:

- Email verification code
- Time-based authentication apps

Flow:

text

User logs in → System sends verification code → User enters code → Access granted

Libraries: pyotp, Flask-Mail

8. Secure Session Management

Sessions are configured with security flags.

Implementation:

python

```
app.config['SESSION_COOKIE_HTTPONLY'] = True    # No JavaScript access
app.config['SESSION_COOKIE_SAMESITE'] = 'Lax'   # CSRF protection
app.config['PERMANENT_SESSION_LIFETIME'] = 7200 # 2 hour timeout
```

Purpose: Prevents session hijacking and automatically expires sessions.

9. Input Validation

User inputs are validated before processing.

Implementation:

python

```
if len(password) < 8:
   flash("Password too short")
if not allowed_file(file.filename):
   flash("Invalid file type")
if not re.match(email_pattern, email):
   flash("Invalid email format")
```

Purpose: Ensures data integrity and prevents malformed input from causing issues.

10. Graceful Error Handling

Instead of crashing, the application shows safe error messages.

Implementation:

python

```
try:
    # Database operation
except sqlite3.Error:
    flash("Database error. Please try again.")
    return redirect(url_for("index"))
```

Purpose: Prevents leaking sensitive system information to users.

11. Static Application Security Testing (SAST)

Code scanning tools automatically detect vulnerabilities.

Tools used:

- Bandit (Python security scanner)
- Manual code review

Purpose: Automatically detects security issues before deployment.

12. Sandboxing

The application runs inside a Docker container.

Benefits:

- Isolated environment
- Prevents system-level compromise
- Controlled resource usage

Command:

bash

```
docker run -p 5000:5000 pizza-blog-app
```

13. Monitoring and Incident Response

Security monitoring tracks suspicious activity.

What we monitor:

- Login attempt logs
- IP addresses
- Failed authentication events

Response actions:

- Log the event for forensic analysis
- Block suspicious IP addresses
- Alert administrators of unusual patterns

Summary Table

Security Feature	Implementation Status
Password Hashing	✅ bcrypt
HTTPS/SSL	✅ Self-signed certificate
CSRF Protection	✅ Cryptographic tokens
XSS Protection	✅ HTML escaping
SQL Injection	✅ Parameterized queries
Brute Force	✅ 5 attempts, 15-min lockout
Session Security	✅ HTTP-only, SameSite
Input Validation	✅ Length, format, type
Error Handling	✅ Graceful messages
Sandboxing	✅ Docker container

Code Review Process

Two code reviews were performed.

Code Review 1 – Before Security Implementation

Issues identified:

- Hardcoded secret keys
- no rate limiting
- no CSRF protection

Code Review 2 – After Security Implementation

Improvements verified:

- password hashing
- CSRF protection
- login protection
- secure sessions
- Docker sandboxing

Benefits to Society

The Bonnyrigg Pizza Blog benefits the community by:

- providing a platform for sharing cooking knowledge
- supporting local community engagement
- promoting food culture
- ensuring user privacy and security

A secure platform ensures trust, reliability, and safe participation.

Logbook Requirement

Date	Task
Week 1	Original pizza blog development
Week 2	Threat analysis
Week 3	Security architecture design
Week 4	Implement CSRF and hashing
Week 5	Implement 2FA and brute force protection
Week 6	Code review and testing